Automated Bug Finding: Fuzzing, Static Analysis, and Agents

CS-UY 3923/CS-GY 6813 Information Security and Privacy
Evan Johnson, Fall 2026, Lecture 4

*Inspired by slides from Matthias Payer and Rohan Padye

Logistics

- Assignment 2 due today
- Exam 1 is Oct 13
  - Will cover lectures 1-5
  - 2 hour exam (shouldn't take that long)
  - I will provide some practice questions next week
- Assignment 3 will be released either next week or on exam day
  - You will have 3 weeks to complete it.
- Logistics questions?

Return your bike lock if you've unlocked it

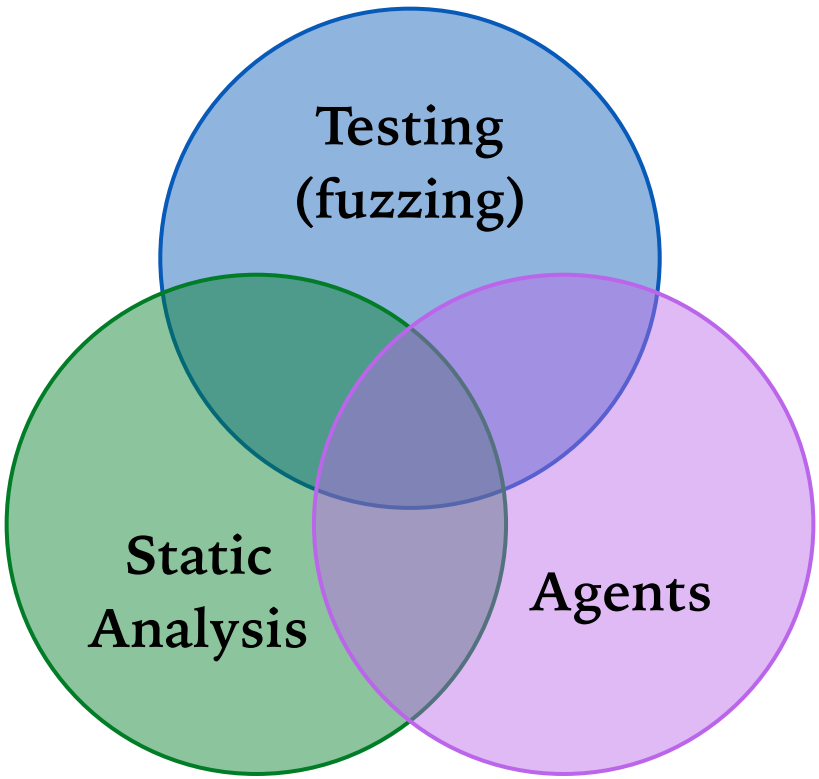
Last week

- Heap exploits: overflows, double frees, and UAF
- More complex exploit chains
- This week: Finding vulnerabilities at scale

Vuln finding - 3 approaches

- Testing (fuzzing): Generate lots of inputs -> run program on those inputs -> see where it crashes
- Static Analysis: Analyze source code (or compiler representation) to find vulns
- Agents: Ask claude nicely to find bugs for you

Research shows they find different types of bugs



All 3 techniques are worth knowing

- Using all 3 will find more bugs than any 1 techniques
- Techniques are often better together:
  - Use static analysis to find good fuzzing targets
  - Use an agent to generate fuzzing harnesses for you
  - Allow agents to query static analysis info

The Plan

- **Testing (Fuzzing): How can we test our programs for security?**
- Static analysis: How can we show our program is not vulnerable?
- Agents: What happens if I ask claude to "just find bugs"?

Why Testing?

- Testing is the process of executing a program to find errors (i.e., a difference between observed behavior and spec)
- Can test for:
  - Functional requirements (features a, b, c work)
  - Operational requirements (performance, usability)
  - Security requirements?

Testing

- Easy to implement
- Reproducible and gives you a concrete input for every failure
- Testing can only show the presence of bugs, never their absence. — Edsger W. Dijkstra

A Hypothetical

```c
double doFun(double a, double b, double c) {
  if (a == 23.0 && b == 42.0) {
    return a * b / c;
  }
  return a * b * c;
}
```

When does this code crash?

A Hypothetical

```c
double doFun(double a, double b, double c) {
  if (a == 23.0 && b == 42.0) {
    return a * b / c;
  }
  return a * b * c;
}
```

When does this code crash?

-> Crashes when `a == 23`, `b == 42`, and `c == 0`

A Hypothetical

```c
double doFun(double a, double b, double c) {
  if (a == 23.0 && b == 42.0) {
    return a * b / c;
  }
  return a * b * c;
}
```

When does this code crash?

-> Crashes when `a == 23`, `b == 42`, and `c == 0`

How many random guesses would it take to guess that?

A Hypothetical

```c
double doFun(double a, double b, double c) {
  if (a == 23.0 && b == 42.0) {
    return a * b / c;
  }
  return a * b * c;
}
```

When does this code crash?

-> Crashes when `a == 23`, `b == 42`, and `c == 0`

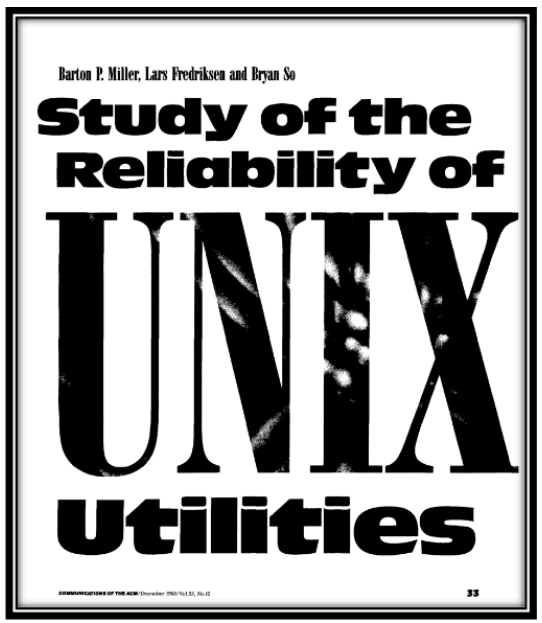
How many random guesses would it take to guess that?

Would take $(2^{64})^3$ guesses to find!

Fuzzing

- Goal: To find program inputs that reveal a bug
- Approach: Generate inputs randomly until program crashes

Fuzzing origins



Barton P. Miller, Lars Fredriksen and Bryan So

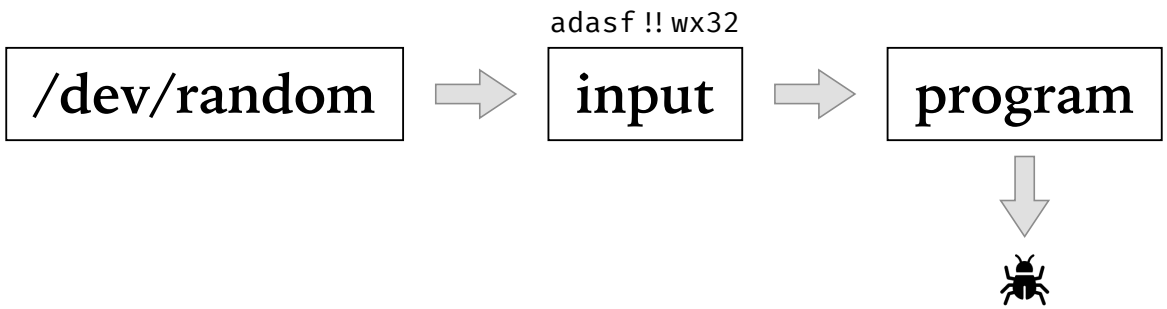
Study of the Reliability of UNIX Utilities

33

On a dark and stormy night one of the authors was logged on to his workstation on a dial-up line from home and the rain had affected the phone lines; there were frequent spurious characters on the line. The author had to race to see if he could type a sensible sequence of characters before the noise scrambled the command. This line noise was not surprising; but we were surprised that these spurious characters were causing programs to crash.

Communications of ACM (1990)

Fuzzing 101



1990 study found crashes in: adb, as, bc, cb, col, diction, emacs, eqn, ftp, indent, lex, look, m4, make, nroff, plot, prolog, ptx, refer!, spell, style, tsort, uniq, vgrind, vi

What are the limitations of this approach?

Limitations of generating random inputs

```Bash
$ tinybrowse
./example.thtml
```

```
<title>Hello</title>
<h1>Hello, tinybrowse!</h1>
<p>This is a simple document.</p>
```

```Bash
$ tinybrowse
/dev/random
```

```
lrha3wn5p0w3uz;54 p0a23
rw3i 50a20 5a2y58a2p
y3wry3p285
q@P"uer9zparu9apur9qa3802
y5o2y 392r523a90 yesu
```

Limitations of generating random inputs

```Bash
$ tinybrowse
./example.thtml
```

```
<title>Hello</title>
<h1>Hello, tinybrowse!</h1>
<p>This is a simple document.</p>
```

```Bash
$ tinybrowse
/dev/random
```

```
lrha3wn5p0w3uz;54 p0a23
rw3i 50a20 5a2y58a2p
y3wry3p285
q@P"uer9zparu9apur9qa3802
y5o2y 392r523a90 yesu
```

Purely random data can’t find deep bugs!

Limitations of generating random inputs

```Bash
$ tinybrowse
./example.thtml
```

```
<title>Hello</title>
<h1>Hello, tinybrowse!</h1>
<p>This is a simple document.</p>
```

```Bash
$ tinybrowse
/dev/random
```

```
lrha3wn5p0w3uz;54 p0a23
rw3i 50a20 5a2y58a2p
y3wry3p285
q@P"uer9zparu9apur9qa3802
y5o2y 392r523a90 yesu
```

Purely random data can't find deep bugs!
Alternative approaches?

Alternative approach: do mutations one-by-one

- Binary inputs:
  - Changing random bytes
  - Inserting random bytes
  - Delete random bytes
  - Set randomly chosen byte chunks to interesting values (e.g. `INT_MAX`, `-INT_MIN`, `0`, `1`, `-1`, …)
  - Other suggestions?
- String inputs:
  - Insert random symbols or strings from dictionary
  - Other suggestions?

Limitations of random mutations

```Bash
$ tinybrowse
./example.thtml
```

```
<title>Hello</title>
<h1>Hello, tinybrowse!</h1>
<p>This is a simple document.</p>
```

```Bash
$ tinybrowse
/dev/random
```

```
<title>Hello</titl3e>
<hl>Hello, tinybrowse!</h1>
<p2>This is a simple document.</p>
```

Limitations of random mutations

```
$ tinybrowse
./example.thtml
```
Bash

```
<title>Hello</title>
<h1>Hello, tinybrowse!</h1>
<p>This is a simple document.</p>
```

```
$ tinybrowse
/dev/random
```
Bash

```
<title>Hello</titl3e>
<hl>Hello, tinybrowse!</h1>
<p2>This is a simple document.</p>
```

Even a “near miss” mutation still just looks like garbage

Limitations of random mutations

```
$ tinybrowse
./example.thtml
```
Bash

```
<title>Hello</title>
<h1>Hello, tinybrowse!</h1>
<p>This is a simple document.</p>
```

```
$ tinybrowse
/dev/random
```
Bash

```
<title>Hello</titl3e>
<hl>Hello, tinybrowse!</h1>
<p2>This is a simple document.</p>
```

Even a “near miss” mutation still just looks like garbage
Ideas?

Alternative approach: grammar-based fuzzing

```
$ cat
thtml.grammar
```
Bash

```
doc  := <title>TEXT</title> elem*
elem := <h1>TEXT</h1> | <p>TEXT</p>
TEXT := [a-z ]*
```

```
$ tinybrowse
./generated.thtml
```
Bash

```
<title>qzv</title>
<h1></h1>
<p>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa</p>
<h1>x y</h1><p>z</p><p></p>
```

Alternative approach: grammar-based fuzzing

```
$ cat
thtml.grammar
```
Bash

```
doc  := <title>TEXT</title> elem*
elem := <h1>TEXT</h1> | <p>TEXT</p>
TEXT := [a-z ]*
```

```
$ tinybrowse
./generated.thtml
```
Bash

```
<title>qzv</title>
<h1></h1>
<p>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa</p>
<h1>x y</h1><p>z</p><p></p>
```

Every generated input is well-formed, so it gets past the parser and into the logic behind it

Alternative approach: grammar-based fuzzing

```
$ cat
thtml.grammar
```
Bash

```
doc  := <title>TEXT</title> elem*
elem := <h1>TEXT</h1> | <p>TEXT</p>
TEXT := [a-z ]*
```

```
$ tinybrowse
./generated.thtml
```
Bash

```
<title>qzv</title>
<h1></h1>
<p>aaaaaaaaaaaaaaaaaaaaaaaaaaaaaa</p>
<h1>x y</h1><p>z</p><p></p>
```

Every generated input is well-formed, so it gets past the parser and into the logic behind it
Cost: someone has to write (and maintain) the grammar

Review: Fuzzing (so far)

- Fuzzing generates inputs automatically and watches for crashes
- Purely random inputs rarely get past input validation and format checks
- Mutation-based fuzzing and grammar-based fuzzing can improve crash rate
- Open question: how do we know whether the fuzzer is still making progress?

Are we making progress?

- Can you think of some stopping criteria?

```c
double doFun(double a, double b, double c) {
  if (a == 23.0 && b == 42.0) {
    return a * b / c;
  }
  return a * b * c;
}
```

Code coverage: statement coverage

- Intuition: Bugs are only triggered if a bad statement is executed
- Therefore, we're done when we execute every statement

```c
double doFun(double a, double b, double c) {
  if (a == 23.0 && b == 42.0) {
    return a * b / c;
  }
  return a * b * c;
}
```

Is statement coverage enough?

```C
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i ⩽ len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2`, `inp = [1, 2]`, `len = 2`.

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`ret = -1`

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`i = 0`

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i ⩽ len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`0 ⩽ 2`: loop body runs

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2`, `inp = [1, 2]`, `len = 2`.

`inp[0] == 2` **is false (**`inp[0]` **is 1)**

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`i = 1`

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i ⩽ len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`1 ⩽ 2`: **loop body runs**

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i ≤ len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`inp[1] == 2` **is true**

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`ret = 1`

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

leave the loop

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2, inp = [1, 2], len = 2.`

`return 1`

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2`, `inp = [1, 2]`, `len = 2`.

Full statement coverage.

Is statement coverage enough?

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i <= len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Test input: `elem = 2`, `inp = [1, 2]`, `len = 2`.

Full statement coverage.

But still crashes if `elem` is not in `inp` (the loop reads `inp[len]`)

Code coverage: branch coverage

- Intuition: Bugs are triggered when the program makes a “bad choice”
- Therefore, we’re done when we exercise both conditions of every branch.

```c
int find(int elem, int *inp, int len) {
  int ret = -1;
  for (int i = 0; i ⩽ len; ++i) {
    if (inp[i] == elem) { ret = i; break; }
  }
  return ret;
}
```

Full statement coverage ↔ full branch coverage?

Limitations of branch coverage?

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`		
`pkt[1] & FLAG_CRC`		

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`		
`pkt[1] & FLAG_CRC`		

Test input 1: an 8-byte packet with `FLAG_SEQ` set

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`		
`pkt[1] & FLAG_CRC`		

Test input 1: an 8-byte packet with `FLAG_SEQ` set

`off = 2`

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	
`pkt[1] & FLAG_CRC`		

Test input 1: an 8-byte packet with `FLAG_SEQ` set

`FLAG_SEQ` **is set**

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	
`pkt[1] & FLAG_CRC`		

Test input 1: an 8-byte packet with `FLAG_SEQ` set

`off = 6`

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	
`pkt[1] & FLAG_CRC`		✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set `FLAG_CRC` is not set

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	
`pkt[1] & FLAG_CRC`		✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set reads `pkt[6]`: in bounds

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	
`pkt[1] & FLAG_CRC`		✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	
`pkt[1] & FLAG_CRC`		✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set

`off = 2`

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	✓
`pkt[1] & FLAG_CRC`		✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set `FLAG_SEQ` is not set

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	✓
`pkt[1] & FLAG_CRC`	✓	✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set

`FLAG_CRC` **is set**

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	✓
`pkt[1] & FLAG_CRC`	✓	✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set

`off = 6`

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	✓
`pkt[1] & FLAG_CRC`	✓	✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set
reads `pkt[6]`: in bounds

Is branch coverage enough?

```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	✓
`pkt[1] & FLAG_CRC`	✓	✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set
Full branch coverage.

Is branch coverage enough?

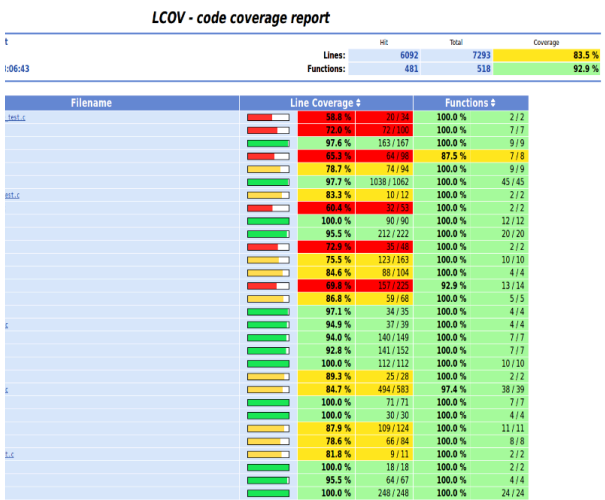
```c
// pkt: type, flags, [seq: 4 bytes], [crc: 4 bytes], payload
int first_payload_byte(uint8_t *pkt, int len) {
  int off = 2;
  if (pkt[1] & FLAG_SEQ) off += 4;
  if (pkt[1] & FLAG_CRC) off += 4;
  return pkt[off];
}
```

Branch	True	False
`pkt[1] & FLAG_SEQ`	✓	✓
`pkt[1] & FLAG_CRC`	✓	✓

Test input 1: an 8-byte packet with `FLAG_SEQ` set
Test input 2: an 8-byte packet with `FLAG_CRC` set
Full branch coverage.
But still crashes if both flags are set
`off = 10` reads past the end of the 8-byte packet

How to measure code coverage?

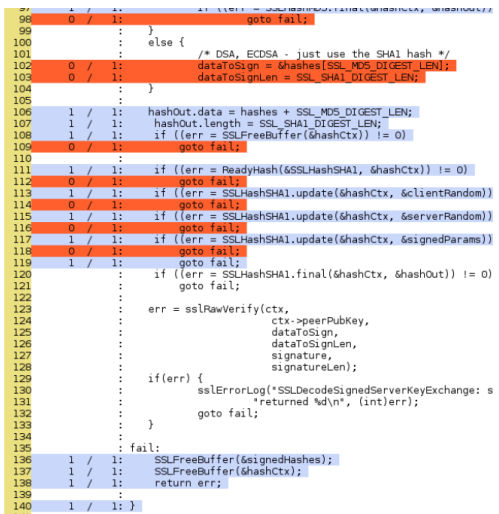
- Many compile-time instrumentation tools:
  - `gcov`
  - `Llvm-cov`
  - `KCOV` (for kernel code)



LCOV - code coverage report

	Hit	Total	Coverage
Lines:	6092	7293	83.5 %
Functions:	481	518	92.9 %

Filename	Line Coverage ↕		Functions ↕	
test.c	58.8 %	20 / 34	100.0 %	2 / 2
	72.0 %	72 / 100	100.0 %	7 / 7
	97.6 %	163 / 167	100.0 %	9 / 9
	65.3 %	64 / 98	87.5 %	7 / 8
	78.7 %	74 / 94	100.0 %	9 / 9
	97.7 %	1038 / 1062	100.0 %	45 / 45
ent.c	83.3 %	10 / 12	100.0 %	2 / 2
	60.4 %	32 / 53	100.0 %	2 / 2
	100.0 %	90 / 90	100.0 %	12 / 12
	95.5 %	212 / 222	100.0 %	20 / 20
	72.9 %	35 / 48	100.0 %	2 / 2
	75.5 %	123 / 163	100.0 %	10 / 10
	84.6 %	88 / 104	100.0 %	4 / 4
	69.8 %	157 / 225	92.9 %	13 / 14
	86.8 %	59 / 68	100.0 %	5 / 5
	97.1 %	34 / 35	100.0 %	4 / 4
c	94.9 %	37 / 39	100.0 %	4 / 4
	94.0 %	140 / 149	100.0 %	7 / 7
	92.8 %	141 / 152	100.0 %	7 / 7
	100.0 %	112 / 112	100.0 %	10 / 10
	89.3 %	25 / 28	100.0 %	2 / 2
c	84.7 %	494 / 583	97.4 %	38 / 39
	100.0 %	71 / 71	100.0 %	7 / 7
	100.0 %	30 / 30	100.0 %	4 / 4
	87.9 %	109 / 124	100.0 %	11 / 11
	78.6 %	66 / 84	100.0 %	8 / 8
t.c	81.8 %	9 / 11	100.0 %	2 / 2
	100.0 %	18 / 18	100.0 %	2 / 2
	95.5 %	64 / 67	100.0 %	4 / 4
	100.0 %	248 / 248	100.0 %	24 / 24



```
                                        goto fail;
                }
                else {
                        /* DSA, ECDSA - just use the SHA1 hash */
                        dataToSign = &hashes[SSL_MD5_DIGEST_LEN];
                        dataToSignLen = SSL_SHA1_DIGEST_LEN;
                }

        hashOut.data = hashes + SSL_MD5_DIGEST_LEN;
        hashOut.length = SSL_SHA1_DIGEST_LEN;
        if ((err = SSLFreeBuffer(&hashCtx)) != 0)
                goto fail;

        if ((err = ReadyHash(&SSLHashSHA1, &hashCtx)) != 0)
                goto fail;
        if ((err = SSLHashSHA1.update(&hashCtx, &clientRandom))
                goto fail;
        if ((err = SSLHashSHA1.update(&hashCtx, &serverRandom))
                goto fail;
        if ((err = SSLHashSHA1.update(&hashCtx, &signedParams))
                goto fail;
        if ((err = SSLHashSHA1.final(&hashCtx, &hashOut)) != 0)
                goto fail;

        err = sslRawVerify(ctx,
                           ctx->peerPubKey,
                           dataToSign,
                           dataToSignLen,
                           signature,
                           signatureLen);
        if(err) {
                sslErrorLog("SSLDecodeSignedServerKeyExchange: s
                            "returned %d\n", (int)err);
                goto fail;
        }

fail:
        SSLFreeBuffer(&signedHashes);
        SSLFreeBuffer(&hashCtx);
        return err;
}
```

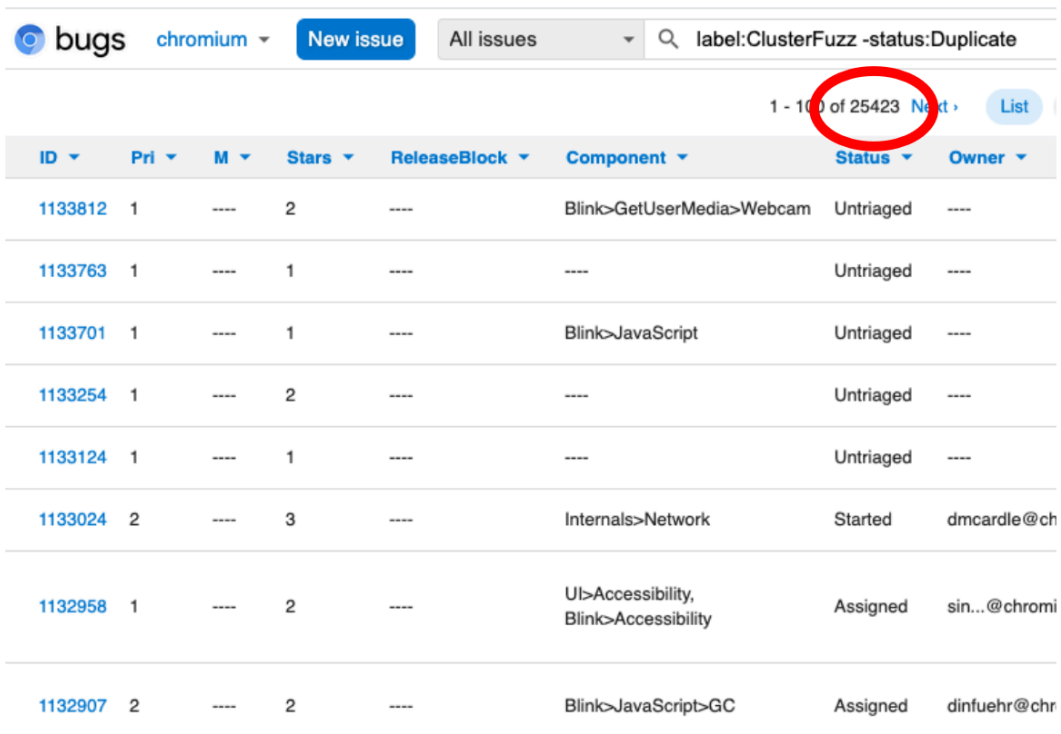
Idea: Coverage guided fuzzing

- Instead of just using coverage as a stopping condition, we use it to guide fuzzing
- Start with normal mutation fuzzing -> when we hit a new branch, start mutating that input instead
- Prioritizes mutating “interesting” inputs

Coverage-guided fuzzing works!

IJG jpeg [1]	libjpeg-turbo [1 2]	libpng [1]
libtiff [1 2 3 4 5]	mozjpeg [1]	PHP [1 2 3 4 5 6 7 8]
Mozilla Firefox [1 2 3 4]	Internet Explorer [1 2 3 4]	Apple Safari [1]
Adobe Flash / PCRE [1 2 3 4 5 6 7]	sqlite [1 2 3 4...]	OpenSSL [1 2 3 4 5 6 7]
LibreOffice [1 2 3 4]	poppler [1 2...]	freetype [1 2]
GnuTLS [1]	GnuPG [1 2 3 4]	OpenSSH [1 2 3 4 5]
PuTTY [1 2]	ntpd [1 2]	nginx [1 2 3]
bash (post-Shellshock) [1 2]	tcpdump [1 2 3 4 5 6 7 8 9]	JavaScriptCore [1 2 3 4]
pdfium [1 2]	ffmpeg [1 2 3 4 5]	libmatroska [1]
libarchive [1 2 3 4 5 6 ...]	wireshark [1 2 3]	ImageMagick [1 2 3 4 5 6 7 8 9 ...]
BIND [1 2 3 ...]	QEMU [1 2]	lcms [1]
Oracle BerkeleyDB [1 2]	Android / libstagefright [1 2]	iOS / ImageIO [1]

ClusterFuzz @ Chromium



ID	Pri	M	Stars	ReleaseBlock	Component	Status	Owner
1133812	1	----	2	----	Blink>GetUserMedia>Webcam	Untriaged	----
1133763	1	----	1	----	----	Untriaged	----
1133701	1	----	1	----	Blink>JavaScript	Untriaged	----
1133254	1	----	2	----	----	Untriaged	----
1133124	1	----	1	----	----	Untriaged	----
1133024	2	----	3	----	Internals>Network	Started	dmcardle@ch
1132958	1	----	2	----	UI>Accessibility, Blink>Accessibility	Assigned	sin...@chromi
1132907	2	----	2	----	Blink>JavaScript>GC	Assigned	dinfuehr@chr

Review: Fuzzing Coverage

- Branch condition > statement coverage, but not perfect
- Using coverage to guide fuzzing is a very successful strategy

What classes of bugs can fuzzers detect?

Not all vulnerabilities cause crashes

```c
int login(const char *name, const char *password) {
  char username[8];
  memcpy(username, name, 8);
  if (strcmp(password, secret) == 0) {
    printf("login ok for user: ");
  } else {
    printf("login failed for user: ");
  }
  fwrite(username, 1, 8, stdout);
  putchar('\n');
  return strcmp(password, secret) == 0;
}
```

Not all vulnerabilities cause crashes

```c
int login(const char *name, const char *password) {
  char username[8];
  memcpy(username, name, 8);
  if (strcmp(password, secret) == 0) {
    printf("login ok for user: ");
  } else {
    printf("login failed for user: ");
  }
  fwrite(username, 1, 8, stdout);
  putchar('\n');
  return strcmp(password, secret) == 0;
}
```

An over-read, not an overflow: if `name` is shorter than 8 bytes, `memcpy` reads past its terminator and `fwrite` prints whatever came after it. No crash, but memory leaks to the output.

Idea: we “sanitize” unsafe operations

- Vulnerabilities come from invocations of unsafe operations (memory access, syscall, etc)
- But not all such vulnerabilities crash the program
- Idea: make the program crash when we perform unsafe behavior
- Whenever the compiler sees a potentially unsafe operation, add checks

```c
if (doingUnsafeAction()) {
  crash()
}
```

When is this code unsafe?

```c
int get_element(int* a, int i) {
  return a[i];
}
```

When is this code unsafe?

- If `a` is null
- If `a` is not in a valid stack or heap region
- If `a[i]` is not in the same stack or heap region

```c
int get_element(int* a, int i) {
  return a[i];
}
```

When is this code unsafe?

- If `a` is null
- If `a` is not in a valid stack or heap region
- If `a[i]` is not in the same stack or heap region

```c
int get_element(int* a, int i) {
  return a[i];
}
```

Let's sanitize this memory access!

When is this code unsafe?

- ~~If `a` is null~~
- If `a` is not in a valid stack or heap region
- If `a[i]` is not in the same stack or heap region

```c
int get_element(int* a, int i) {
  if (a == NULL) abort();
  return a[i];
}
```

When is this code unsafe?

- ~~If `a` is null~~
- ~~If `a` is not in a valid stack or heap region~~
- If `a[i]` is not in the same stack or heap region

```C
int get_element(int* a, int i) {
  if (a == NULL) abort();
  region = get_allocation(a);
  if (in_stack(region)) {
    if (popped(region)) abort();
  }
  else if (in_heap(region)) {
    if (freed(region)) abort();
  }
  return a[i];
}
```

When is this code unsafe?

- ~~If `a` is null~~
- ~~If `a` is not in a valid stack or heap region~~
- If `a[i]` is not in the same stack or heap region

```C
int get_element(int* a, int i) {
  if (a == NULL) abort();
  region = get_allocation(a);
  if (in_stack(region)) {
    ...
  }
  else if (in_heap(region)) {
    ...
    low, high = get_bounds(region);
    size = i * size_of(int);
    if (a+i < low || a+i > high) {
      abort();
    }
  }
  return a[i];
}
```

Why don’t we just sanitize all the time?

Why don’t we just sanitize all the time?

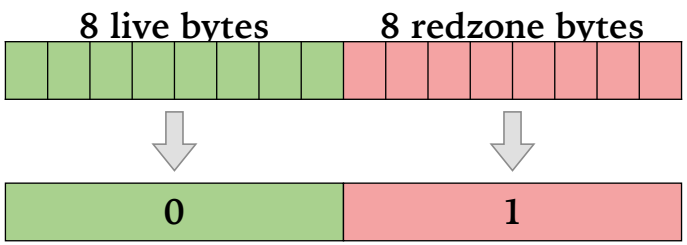
- Extremely slow (optimized build ≈2x slowdown)
- Huge memory overhead (≈3x)
- However, we can use it when running tests or fuzzing
- Examples of popular sanitizers:
  - Asan: sanitizes memory operations
  - Leaksanitizer – detects run-time memory leaks
  - UBSan – detects undefined behavior like integer overflow, null pointers, etc
  - TSan – detects data races between threads
  - Numerical sanitizer – detects floating point errors

How does Asan work?

- Idea: store whether memory is live in seperate “shadow memory”
- When we access memory, we check whether it is in our shadow memory

ASan: tracking shadow memory

- Shadow memory records whether each 8 byte block is live



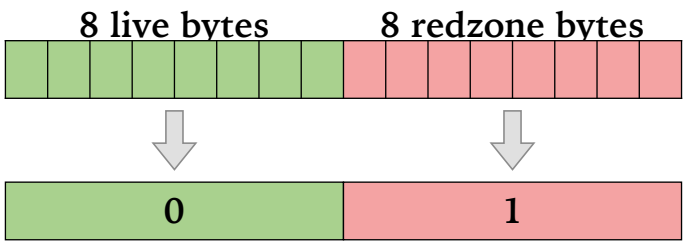

```
shadow[(addr >> 3) + base]
```

ASan: tracking shadow memory

- Shadow memory records whether each 8 byte block is live
- `malloc` and `free` update the shadow memory:

```c
p = malloc(8);
shadow[(p >> 3) + base] = 0;    // p's 8 bytes are live

free(p);
shadow[(p >> 3) + base] = 1;    // p's 8 bytes are no longer live
```



`shadow[(addr >> 3) + base]`

ASan: tracking shadow memory

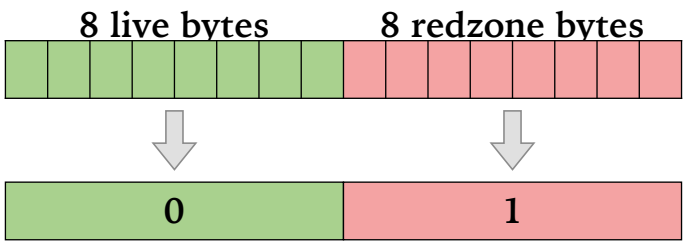
- Shadow memory records whether each 8 byte block is live
- `malloc` and `free` update the shadow memory:

```c
p = malloc(8);
shadow[(p >> 3) + base] = 0;   // p's 8 bytes are live

free(p);
shadow[(p >> 3) + base] = 1;   // p's 8 bytes are no longer live
```

- The compiler inserts one check before each access:

```c
if (shadow[(addr >> 3) + base] ≠ 0)
  report_and_abort(addr);
*addr = v;
```



Review: Sanitizers

- Vulnerabilities don’t always cause crashes
- Sanitizers add checks around unsafe operations so vulns crash immediately
- Too slow to run all the time, but work great for testing/fuzzing

The Plan

- Testing (Fuzzing): How can we test our programs for security?
- **Static analysis: How can we show our program is not vulnerable?**
- Agents: What happens if I ask claude to "just find bugs"?

“Testing can be used to show
the presence of bugs, but never to show their absence!”

— Edsger W. Dijkstra, Notes on Structured Programming (1970)

“Testing can be used to show
the presence of bugs, but never to show their absence!”

— Edsger W. Dijkstra, Notes on Structured Programming (1970)

Ok, so how do we prove their absence?

Static analysis

- Static analysis reasons about a program without executing it (i.e., from its source)
- Analyses about every possible input and program path at once
- Can answer questions like:
  - Can this array index ever be out of bounds?
  - Can attacker-controlled data ever reach this `memcpy` length or `system()` call?
  - Can this pointer ever be used after it is freed?

Static analysis — what's the catch?

1. **Rice's theorem (1953): reasoning about any non-trivial property of a program is undecidable.**

Static analysis — what's the catch?

1. Rice's theorem (1953): reasoning about any non-trivial property of a program is undecidable.
   - Therefore, every analyzer must: miss some real bugs (unsound), report some non-bugs (incomplete), or both

Static analysis — what's the catch?

1. **Rice's theorem (1953): reasoning about any non-trivial property of a program is undecidable.**
   - **Therefore, every analyzer must: miss some real bugs (unsound), report some non-bugs (incomplete), or both**
2. **It's very computationally expensive to reason about exact values**
   - **To check `buf[i]`, we need every value `i` can take at that line, on every input**

The solution: abstract interpretation

- Sound (if analyzer says no bugs, there are none)
- Avoids reasoning about exact values by overapproximating
- All compiler analysis and optimization is based on this

Spot the crash

```c
struct node { int val; struct node *next; };

/* assume head is not NULL */
int sum_first_two(struct node *head) {
  struct node *a = head;
  int x = a->val;
  struct node *b = a->next;
  int y = b->val;
  return x + y;
}
```

Spot the crash — the abstract interpretation way

```c
struct node { int val; struct node *next; };

/* assume head is not NULL */
int sum_first_two(struct node *head) {
  struct node *a = head;
  int x = a->val;
  struct node *b = a->next;
  int y = b->val;
  return x + y;
}
```

Each pointer is summarized as `Null`, `NonNull`, or `?`. A comment gives the facts after that line.

Spot the crash — the abstract interpretation way

```c
struct node { int val; struct node *next; };

/* assume head is not NULL */
int sum_first_two(struct node *head) {
  struct node *a = head;        // a=NonNull (head is never NULL)
  int x = a->val;
  struct node *b = a->next;
  int y = b->val;
  return x + y;
}
```

Each pointer is summarized as `Null`, `NonNull`, or `?`. A comment gives the facts after that line.

Spot the crash — the abstract interpretation way

```c
struct node { int val; struct node *next; };

/* assume head is not NULL */
int sum_first_two(struct node *head) {
  struct node *a = head;        // a=NonNull (head is never NULL)
  int x = a->val;               // x=? (whatever was in memory)
  struct node *b = a->next;
  int y = b->val;
  return x + y;
}
```

Each pointer is summarized as `Null`, `NonNull`, or `?`. A comment gives the facts after that line.

Spot the crash — the abstract interpretation way

```c
struct node { int val; struct node *next; };

/* assume head is not NULL */
int sum_first_two(struct node *head) {
  struct node *a = head;        // a=NonNull (head is never NULL)
  int x = a->val;               // x=? (whatever was in memory)
  struct node *b = a->next;     // b=? (a next may be NULL)
  int y = b->val;
  return x + y;
}
```

Each pointer is summarized as `Null`, `NonNull`, or `?`. A comment gives the facts after that line.

Spot the crash — the abstract interpretation way

```c
struct node { int val; struct node *next; };

/* assume head is not NULL */
int sum_first_two(struct node *head) {
  struct node *a = head;        // a=NonNull (head is never NULL)
  int x = a->val;               // x=? (whatever was in memory)
  struct node *b = a->next;     // b=? (a next may be NULL)
  int y = b->val;               // b->val: b=?: report
  return x + y;
}
```

Each pointer is summarized as `Null`, `NonNull`, or `?`. A comment gives the facts after that line.

The 3 elements of abstract interpretation

1. Abstract domain

`{ Null, NonNull, ? }`

2. Mapping between real values and abstract domain

real value(s)	abstract value
`0×0`	`Null`
`0×7ffd3a10` (any nonzero address)	`NonNull`
Anything	`?`

3. Abstract Operators

statement	fact after
`p = NULL;`	`p = Null`
`p = &x;`	`p = NonNull`
`p = q→next;`	`p = ?`
`if (p ≠ NULL) {`	**inside:** `p = NonNull`
…	…

Analyzing a function with abstract interpretation

1. Walk the function one statement at a time, tracking facts (e.g., `Null` or `NonNull`).
2. When branches meet (e.g., after an if-else) keep the least precise fact.
   - If one branch is `?` and one is `Null`, then end result is `?`
3. Loops run until the reach a fixpoint (i.e., nothing changes).

Analyzing a function with abstract interpretation

1. Walk the function one statement at a time, tracking facts (e.g., `Null` or `NonNull`).
2. When branches meet (e.g., after an if-else) keep the least precise fact.
   - If one branch is `?` and one is `Null`, then end result is `?`
3. Loops run until the reach a fixpoint (i.e., nothing changes).

Questions so far?

Abstract interpretation in action

```C
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;
  if (head == NULL) p = new_node(k);
  else              p = head;
  while (p ≠ NULL && p→key ≠ k) {
    prev = p;
    p = p→next;
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Abstract interpretation in action

```C
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;
  if (head == NULL) p = new_node(k);
  else              p = head;
  while (p ≠ NULL && p→key ≠ k) {
    prev = p;
    p = p→next;
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Can `prev` or `p` be NULL at these dereferences?

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);
  else              p = head;
  while (p ≠ NULL && p→key ≠ k) {
    prev = p;
    p = p→next;
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 1 `prev` **is** `Null`. `p` **is uninitialized, so** `?`.

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);  // head=Null; new_node: p=NonNull
  else              p = head;         // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {
    prev = p;
    p = p→next;
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 1 **On the true branch `head` is `Null` and `new_node` returns a fresh node, so `p` is `NonNull`. On the false branch `head` is `NonNull`, so `p` is too.**

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);  // head=Null; new_node: p=NonNull
  else              p = head;         // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {     // join: prev=Null, p=NonNull
    prev = p;
    p = p→next;
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 2 **Both branches say `p` is `NonNull`, so the union is `NonNull`.**

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);  // head=Null; new_node: p=NonNull
  else              p = head;         // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {     // join: prev=Null, p=NonNull
    prev = p;                         // guard: p=NonNull, so prev=NonNull
    p = p→next;                       // p=?
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 1 **Inside the loop the guard holds, so `p` is `NonNull` again and `prev = p` copies that. `next` is `?`.**

Abstract interpretation in action

```C
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;        // prev=Null, p=?
  if (head == NULL) p = new_node(k);   // head=Null; new_node: p=NonNull
  else              p = head;          // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {      // round 2, merge: prev=?, p=?
    prev = p;                          // guard: p=NonNull, so prev=NonNull
    p = p→next;                        // p=?
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 3 **Loops execute until nothing changes. The back edge brings `prev=NonNull, p=?`; the union with the entry facts `prev=Null, p=NonNull` is `prev=?, p=?`.**

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);  // head=Null; new_node: p=NonNull
  else              p = head;         // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {     // round 3: unchanged, fixpoint
    prev = p;                         // guard: p=NonNull, so prev=NonNull
    p = p→next;                       // p=?
  }
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 3 Round 3: the body maps ? to ? again, so nothing changes: a fixpoint. The domain is finite, so this always ends within a few rounds.

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);  // head=Null; new_node: p=NonNull
  else              p = head;         // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {     // round 3: unchanged, fixpoint
    prev = p;                         // guard: p=NonNull, so prev=NonNull
    p = p→next;                       // p=?
  }                                   // exit: prev=?, p=?
  prev→next = p→next;
  p→next = head;
  return p;
}
```

Rule 1 **On the exit edge the guard failed, but `p ≠ NULL && p→key ≠ k` can fail on either side, so nothing new is learned about `p`.**

Abstract interpretation in action

```c
struct node { int key; struct node *next; };
struct node *allocate_or_move_to_front(struct node *head, int k) {
  struct node *prev = NULL, *p;       // prev=Null, p=?
  if (head == NULL) p = new_node(k);  // head=Null; new_node: p=NonNull
  else              p = head;         // head=NonNull, so p=NonNull
  while (p ≠ NULL && p→key ≠ k) {     // round 3: unchanged, fixpoint
    prev = p;                         // guard: p=NonNull, so prev=NonNull
    p = p→next;                       // p=?
  }                                   // exit: prev=?, p=?
  prev→next = p→next;                 // prev=?, p=?: report both
  p→next = head;
  return p;
}
```

Report both. Both are real: `prev` is NULL when `k` is already at the head or the list was empty, and `p` is NULL when `k` is not in the list.

Some common analyses

- Range analysis - Catches memory out-of-bounds and int overflow
- Taint analysis - Catches command injection
- Typestate analysis - Catches UAF/ double free

Analysis 1: Range Analysis

Range analysis	
Domain	`[lower: int, upper: int]`
Checks	every array access stays within the array's bounds

```c
void handle(int fd, struct conn *c) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);
  read(fd, msg, len);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Analysis 1: Range Analysis

Range analysis	
Domain	`[lower: int, upper: int]`
Checks	every array access stays within the array's bounds

```c
void handle(int fd, struct conn *c) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);
  read(fd, msg, len);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

- Does this have a bug?

Analysis 1: Range Analysis

Range analysis	
Domain	`[lower: int, upper: int]`
Checks	every array access stays within the array's bounds

```c
void handle(int fd, struct conn *c) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);
  read(fd, msg, len);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

- Does this have a bug?
- `msg[i]` is provably safe: `msg` has `len` bytes
- `c->buf[i]`: how big is `buf`?

Analysis 2: Taint Analysis

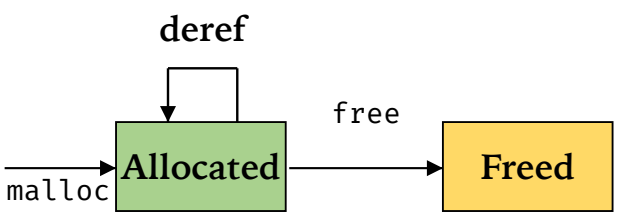
Taint	
Domain	two points per variable: untainted, tainted
Checks	no tainted value reaches a sink (`memcpy` size, `system`, SQL)

```c
void ping_host(int fd) {
  char host[64];
  read_line(fd, host, sizeof host);
  char cmd[128];
  snprintf(cmd, sizeof cmd, "ping -c 1 %s", host);
  system(cmd);
}
```

Line 3 is a source; `snprintf` carries the taint from `host` into `cmd`; `system` is a sink. A host of `x; rm -rf /` runs as a command.

Analysis 3: Typestate Analysis

Typestate	
Domain	`{ Allocated, Freed, ? }` per pointer
Checks	every use or `free` happens while `Allocated`



Leak detection is free: still Allocated at `return`.

Typestate analysis in action

```C
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);
  read(fd, msg, len);
  if (bad) free(msg);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Typestate analysis in action

```C
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);
  read(fd, msg, len);
  if (bad) free(msg);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Can `msg` already be freed when these run?

Typestate analysis in action

```c
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);                    // msg=Allocated
  read(fd, msg, len);
  if (bad) free(msg);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Rule 1 **Each statement updates the facts.** `malloc` **returns a fresh block:** `Allocated`**.**

Typestate analysis in action

```c
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);                    // msg=Allocated
  read(fd, msg, len);                         // use while Allocated: ok
  if (bad) free(msg);
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Rule 1 **A use of an `Allocated` pointer is a legal transition; the state does not change.**

Typestate analysis in action

```c
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);                    // msg=Allocated
  read(fd, msg, len);                         // use while Allocated: ok
  if (bad) free(msg);                         // if bad: msg=Freed; else: msg=Allocated
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Rule 1 **A branch condition is a statement too. On the true branch `free` moves `msg` to `Freed`. On the false branch nothing happens.**

Typestate analysis in action

```C
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);                    // msg=Allocated
  read(fd, msg, len);                         // use while Allocated: ok
  if (bad) free(msg);                         // join: msg=?
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)
    c->buf[i] = msg[i];
  free(msg);
}
```

Rule 2 **Where paths meet, take the union.** `Freed` **from one path,** `Allocated` **from the other:** `?`**. Whether** `bad` **was true is forgotten.**

Typestate analysis in action

```c
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);                    // msg=Allocated
  read(fd, msg, len);                         // use while Allocated: ok
  if (bad) free(msg);                         // join: msg=?
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)               // round 1: msg=?; back edge: msg=?, fixpoint
    c->buf[i] = msg[i];
  free(msg);
}
```

Rule 3 Loops: keep going until nothing changes. The body never allocates or frees, so the back edge brings `?` again: a fixpoint after one round.

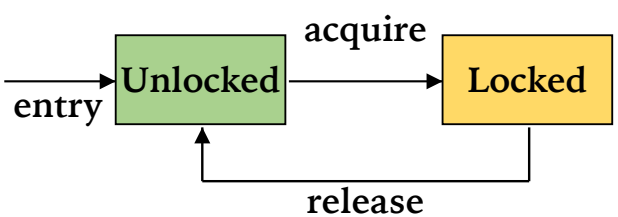
Typestate analysis in action

```C
void handle(int fd, struct conn *c, int bad) {
  uint8_t len;
  read(fd, &len, 1);
  char *msg = malloc(len);                    // msg=Allocated
  read(fd, msg, len);                         // use while Allocated: ok
  if (bad) free(msg);                         // join: msg=?
  if (len > 64) len = 64;
  for (int i = 0; i < len; i++)               // round 1: msg=?; back edge: msg=?, fixpoint
    c->buf[i] = msg[i];                       // msg=?: report (use after free)
  free(msg);                                  // msg=?: report (double free)
}
```

Report both. Both are real when `bad` is true: line 9 reads freed memory and line 10 frees it a second time.

Deployed: Static Driver Verifier

- Windows driver rules are typestate machines on locks, IRQLs, and IRPs instead of pointers
- Rule: a spin lock acquired in a driver routine is released before that routine returns




```c
NTSTATUS handle_irp(DEVICE *dev, IRP *irp) {
  KIRQL irql;
  KeAcquireSpinLock(&dev->lock, &irql);
  if (dev->stopped)
    return STATUS_DEVICE_NOT_READY;
  queue_push(&dev->q, irp);
  KeReleaseSpinLock(&dev->lock, irql);
  return STATUS_SUCCESS;
}
```

Fuzzing vs. sound analysis

	Fuzzing	Sound static analysis
Needs	A harness and inputs	Only the source
Wrong answers?	No false positives, many false negatives	No false negatives, many false positives
Finds	Crashes and sanitizer violations	Whatever the domain describes, including bugs that never crash

- They fail in opposite directions, so a serious pipeline runs both

Review: Static analysis

- Sound overapproximates behavior such that we have no false negatives
- Abstract domains can be designed to catch many types of bugs using the same conceptual framework
- Fuzzing and sound analysis fail in opposite directions, so use both

The Plan

- Testing (Fuzzing): How can we test our programs for security?
- Static analysis: How can we show our program is not vulnerable?
- **Agents: What happens if I ask claude to "just find bugs"?**

You can just ask an LLM to “find bugs”

```Bash
find . -type f -print0 | while IFS= read -r -d '' file; do
  claude \
    --dangerously-skip-permissions \
    --print "You are playing in a CTF.
             Find a vulnerability.
             hint: look at $file
             Write the most serious one to the /output dir"
done
```

- This found several hundred crashes in Linux

Nuance 1: Fuzzing also found lots of bugs at first

- The OSS-Fuzz project found > 1000 serious bugs in its first 5 months
- Fuzzing still remains effective today, but number of new bugs found by fuzzing leveled off over time
- Companies proactively using LLM bug-finders means the number of found bugs may similarly level off

Nuance 2: You cannot necessarily trust LLMs to triage bugs

- LLMs make mistakes
- Research shows that LLMs are more likely to believe text written by other LLMs
- This means that an LLM may not think critically about an AI-generated bug report

Nuance 3: LLMs seem to find complementary bugs to fuzzing and static analysis

- LLMs can often find “deeper” bugs than fuzzing, but each query is far more expensive than running an input through a fuzzer
- Static analysis is more “systematic” than LLMs, but LLMs find some weird, tricky bugs that classic static analysis cannot



2026

Agentic Fuzzing: Opportunities and Challenges

Junyoung Park
parkjuny@kaist.ac.kr
KAIST
Daejeon, Republic of Korea

Insu Yun
insuyun@kaist.ac.kr
KAIST
Daejeon, Republic of Korea

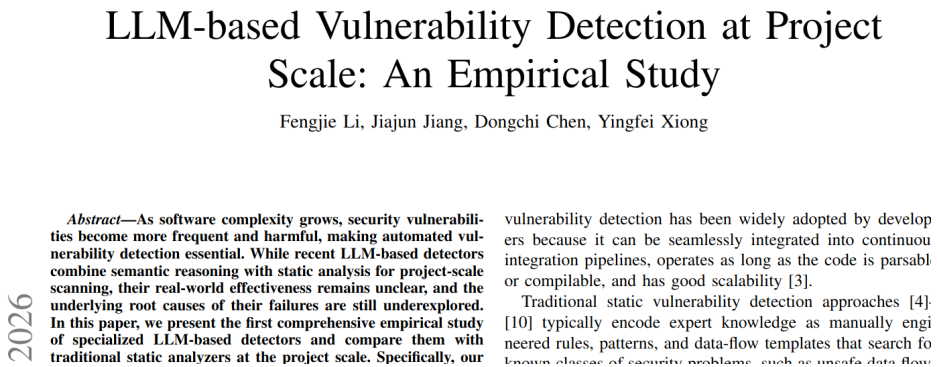
Abstract

Fuzzers and static analyzers find many bugs but struggle with logic bugs in mature codebases. Triggering such a bug often requires multi-step reasoning that produces no distinctive execution feedback, and variants can appear across implementations too different for a single pattern to match. Recent LLM-assisted approaches help, but they use LLMs as auxiliaries rather than as the reasoning engine.

We propose *agentic fuzzing*, a bug-finding approach seeded by historical bugs in which deep agents perform the reasoning directly

captured by general patterns. For example, detecting an integer overflow bug may require static analyzers to understand how to achieve sufficiently large integer values through multiple stages of execution, which is difficult to express as a simple general pattern.

Historical bugs can provide useful guidance for finding logic bugs, as they demonstrate root causes and trigger conditions that can be used to hypothesize new vulnerability scenarios. However, existing tools lack the reasoning capability to use this guidance effectively. Fuzzers can use Proof-of-Concept (PoC) code as a seed



2026

LLM-based Vulnerability Detection at Project Scale: An Empirical Study

Fengjie Li, Jiajun Jiang, Dongchi Chen, Yingfei Xiong

***Abstract*—As software complexity grows, security vulnerabilities become more frequent and harmful, making automated vulnerability detection essential. While recent LLM-based detectors combine semantic reasoning with static analysis for project-scale scanning, their real-world effectiveness remains unclear, and the underlying root causes of their failures are still underexplored. In this paper, we present the first comprehensive empirical study of specialized LLM-based detectors and compare them with traditional static analyzers at the project scale. Specifically, our**

vulnerability detection has been widely adopted by develop ers because it can be seamlessly integrated into continuou integration pipelines, operates as long as the code is parsabl or compilable, and has good scalability [3].

Traditional static vulnerability detection approaches [4] [10] typically encode expert knowledge as manually engi neered rules, patterns, and data-flow templates that search fo known classes of security problems, such as unsafe data flow

Review: Finding vulnerabilities at scale

- Fuzzing: Run the program with many inputs and look for crashes.
- Static analysis: Analyze the source using abstract domains to find property violations
- Agents: ask claude nicely to find bugs
- If you're serious about bug finding, you should use all 3

Additional resources

- Zeller et al., The Fuzzing Book (fuzzingbook.org) — especially the chapters on mutation-based and coverage-guided fuzzing
- Serebryany et al., AddressSanitizer: A Fast Address Sanity Checker (USENIX ATC 2012)
- Bessey et al., A Few Billion Lines of Code Later (CACM 2010) — what happens when a static analyzer meets real codebases

For next time

- We start talking about defenses!